

注意二分查找中的循环不变量（只针对有序数组）^{OBJ}左边应该始终指向无法满足要求的一项，^{OBJ}右边应该始终指向满足要求的一方^{OBJ}当答案空间有范围 + 判定函数具有单调性 → 可以二分答案

5	7	7	8	8	10
---	---	---	---	---	----

X X X ✓
 $O(n)$

问题：返回有序数组中第一个 ≥ 8 的数的位置
如果所有数都 < 8 ，返回数组长度

暴力做法：遍历每个数，询问它是否 ≥ 8 ？

5	7	7	8	8	10
---	---	---	---	---	----

L M R
 $M = \lfloor \frac{L+R}{2} \rfloor$
 $L=0$ $R=n-1=5$

高效做法：

L 和 R 分别指向询问的左右边界，即闭区间 $[L, R]$

M 指向当前正在询问的数

红色背景表示 false，即 < 8

蓝色背景表示 true，即 ≥ 8

白色背景表示不确定

5	7	7	8	8	10
---	---	---	---	---	----

L M R

继续询问

5	7	7	8	8	10
---	---	---	---	---	----

R L

关键：循环不变量

$L - 1$ 始终是红色

$R + 1$ 始终是蓝色

根据循环不变量， $R + 1$ 是我们要找的答案

由于循环结束后 $R + 1 = L$

所以答案也可以用 L 表示

34. 在排序数组中查找元素的第一个和最后一个位置

中等

🔖 相关标签

🏢 相关企业

Ax

给你一个按照非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。请你找出给定目标值在数组中的开始位置和结束位置。

如果数组中不存在目标值 `target`，返回 `[-1, -1]`。

你必须设计并实现时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1:

输入: `nums = [5,7,7,8,8,10]`, `target = 8`

输出: `[3,4]`

示例 2:

输入: `nums = [5,7,7,8,8,10]`, `target = 6`

输出: `[-1,-1]`

示例 3:

输入: `nums = []`, `target = 0`

输出: `[-1,-1]`

提示:

• `0 <= nums.length <= 105`

• `-109 <= nums[i] <= 109`

• `nums` 是一个非递减数组

• `-109 <= target <= 109`

```
class Solution(object):
    def lower_bound(self, nums, target):
        left = 0
        right = len(nums) - 1 #
```

注意我们此时使用的是闭区间

```

[left, right]
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] < target:
            left = mid + 1 #[mid +1, right]
        else:
            right = mid -1 #[left, mid -1]
    return left
def lower_bound2(self, nums, target):
    left = 0
    right = len(nums) #

```

注意我们此时使用的是左闭右开区间 `[left, right)`

```

    while left < right:
        mid = (left + right) // 2
        if nums[mid] < target:
            left = mid + 1 #[mid +1, right

```

)

```

        else:
            right = mid #[left, mid

```

)

```

    return left
#

```

最推荐

```

def lower_bound3(self, nums, target):
    left = -1 #

```

注意我们要左开

```

    right = len(nums) #

```

注意我们此时使用的是左开右开区间 `(left, right)`

```
while left
```

- 1 \< right: #+1 是为了避免死循环

```
mid = (left + right) // 2
if nums[mid] \< target:
```

注意在这里左边应该始终满足判断条件，右边相反

```
left = mid #
```

```
(mid , right )
```

```
else:
    right = mid #
```

```
(left, mid )
```

```
return right
```

关于第三个解法，python3中有一个内置函数，也就是`bisect_left(arr,target)`返回第一个 $\geq$ target的数：

```
class Solution(object):
    def findTheDistanceValue(self, arr1, arr2, d):
        """
        :type arr1: List[int]
        :type arr2: List[int]
        :type d: int
        :rtype: int
        """
        arr2.sort()
        ans = 0
        for x in arr1:
            i = bisect_left(arr2, x-d)
            if i == len(arr2) or arr2[i] \> x+d:
                ans += 1
        return ans
```

bisect_left:

[1, |3, 3, 3, 5]
↑ 插这里 (最左)



bisect_right (顺便记) :

[1, 3, 3, 3|, 5]
↑ 插这里 (最右)





```
def searchRange(self, nums, target):  
    """  
    :type nums: List[int]  
    :type target: int  
    :rtype: List[int]  
    """  
    start = self.lower_bound(nums, target)  
    if start == len(nums) or nums[start] != target:  
        return [-1, -1]  
    end = self.lower_bound(nums, target+1) - 1  
    return [start, end]
```



学习提醒】建议先完成本节的课后作业，再开始下一节的学习！加油！

34. 在排序数组中查找元素的第一个和最后一个位置 <https://leetcode.cn/problems/find-first-and-last-position-of-element-in-sorted-array/solution/er-fen-cha-zhao-zong-shi-xie-bu-dui-yi-g-t9l9/>

课后作业： 2529. 正整数和负整数的最大计数 <https://leetcode.cn/problems/maximum-count-of-positive-integer-and-negative-integer/>

2300. 咒语和药水的成功对数 <https://leetcode.cn/problems/successful-pairs-of-spells-and-potions/>

1385. 两个数组间的距离值 <https://leetcode.cn/problems/find-the-distance-value-between-two-arrays/>

2080. 区间内查询数字的频率 <https://leetcode.cn/problems/range-frequency-queries/>

2563. 统计公平数对的数目 <https://leetcode.cn/problems/count-the-number-of-fair-pairs/>

875. 爱吃香蕉的珂珂 <https://leetcode.cn/problems/koko-eating-bananas/>

2187. 完成旅途的最少时间 <https://leetcode.cn/problems/minimum-time-to-complete-trips/>

275. H 指数 II <https://leetcode.cn/problems/h-index-ii/>

2861. 最大合金数 <https://leetcode.cn/problems/maximum-number-of-alloys/>

2439. 最小化数组中的最大值 <https://leetcode.cn/problems/minimize-maximum-of-array/>

2517. 礼盒的最大甜蜜度 <https://leetcode.cn/problems/maximum-tastiness-of-candy-basket/>

> 来自 <<https://www.bilibili.com/video/BV1AP41137w7?>

[vd_source=35262fcad6990c0926abc68a8932fb15&spm_id_from=333.788.videopod.sections](https://www.bilibili.com/video/BV1AP41137w7?vd_source=35262fcad6990c0926abc68a8932fb15&spm_id_from=333.788.videopod.sections)>